GSD RESEARCH MISSION PACKAGE — FLEET ACTIVATION

GPU Graphics, Compute & GSD-OS Observability Ecosystem

OpenGL/Vulkan · CUDA/OpenCL · SoC Audio · Amiga Expansion
XScreenSaver Engineering · GSD-OS Integration · OTel/Prometheus/Grafana

A Deep Research Study — 1001 Examples, Pure Signal

Vision → Research → Mission
Full Pipeline Execution

Date: March 26, 2026 | Status: Ready for GSD Orchestrator
Activation Profile: Fleet (20 roles) | Est.: 18–24 context windows across 6–8 sessions
Modules: 6 | Waves: 4 | Parallel Tracks: 3

Contents

1 GPU Graphics, Compute & GSD-OS Observability Ecosystem — Vision Guide 4
1.1 Vision . . . 4
1.2 Problem Statement . . . 5
1.3 Core Concept . . . 6
1.3.1 Domain Architecture Map . . . 6
1.3.2 Module Architecture . . . 6
1.4 Research Modules . . . 6
1.4.1 Module 01 — OpenGL/Vulkan Foundations: 1001 Examples Taxonomy . . . 6
1.4.2 Module 02 — Compute Without NVIDIA: CUDA to OpenCL/Vulkan/NEON 7
1.4.3 Module 03 — Amiga Expansion Principles: Zorro Bus and Chipset Architecture . . . 7
1.4.4 Module 04 — XScreenSaver Engineering: GLX, GLSL, and Shadertoy . . . 7
1.4.5 Module 05 — GSD-OS Deep Integration . . . 7
1.4.6 Module 06 — Observability Stack: OpenTelemetry, Prometheus, and Grafana . . . 8
1.5 Chipset Configuration . . . 8
1.6 Scope Boundaries . . . 8
1.6.1 In Scope (v1.0) . . . 8
1.6.2 Out of Scope . . . 9
1.7 Success Criteria . . . 9
1.8 Relationship to Other GSD Documents . . . 10
1.9 The Through-Line . . . 10

2 GPU Graphics, Compute & GSD-OS Observability — Research Reference 11
2.1 How to Use This Document . . . 11
2.2 Module 01 Reference — OpenGL/Vulkan Foundations . . . 11
2.2.1 The NeHe Legacy and the Modern Gap . . . 11
2.2.2 GLSL Version Portability Matrix . . . 12
2.3 Module 02 Reference — Compute Without NVIDIA . . . 13
2.3.1 Raspberry Pi GPU Landscape . . . 13
2.3.2 CUDA to Cross-Vendor Translation . . . 13
2.3.3 Audio DSP on SoC . . . 13
2.4 Module 03 Reference — Amiga Expansion Principles . . . 14
2.4.1 Zorro Bus Architecture . . . 14
2.4.2 Chipset Coprocessor Architecture . . . 14
2.4.3 Modern FPGA Accelerators . . . 14
2.5 Module 04 Reference — XScreenSaver Engineering . . . 14
2.5.1 Framework Architecture . . . 15
2.5.2 Shadertoy Licensing Note . . . 15
2.5.3 50-Hack Curated Subset for GSD-OS . . . 15
2.6 Module 05 Reference — GSD-OS Deep Integration . . . 15

2.7 Module 06 Reference — Observability Stack . . . 16
2.7.1 NVIDIA DCGM Exporter Pipeline . . . 16
2.7.2 Grafana Alloy and Prometheus 3.0 . . . 16
2.7.3 OTel Collector Configuration Skeleton . . . 16
2.8 Safety and Sensitivity Considerations . . . 17
2.9 Source Bibliography . . . 17

3 Milestone Specification **19**
3.1 Mission Objective . . . 19
3.2 Architecture Overview . . . 19
3.3 Deliverables . . . 20
3.4 Component Breakdown . . . 21
3.5 Model Assignment Rationale . . . 22

4 Wave Execution Plan **22**
4.1 Wave Summary . . . 22
4.2 Wave 0 — Foundation . . . 22
4.3 Wave 1 — Parallel Tracks . . . 22
4.4 Wave 2 — Synthesis . . . 23
4.5 Wave 3 — Publication and Verification . . . 24
4.6 Cache Optimization Strategy . . . 24
4.7 Token Budget . . . 24
4.8 Dependency Graph . . . 24

5 Test Plan **25**
5.1 Test Categories . . . 25
5.2 Safety-Critical Tests . . . 25
5.3 Core Functionality Tests (Selected) . . . 26
5.4 Verification Matrix . . . 27

6 Mission Crew Manifest **28**

7 Execution Summary **29**

STAGE 1 — VISION DOCUMENT

1 GPU Graphics, Compute & GSD-OS Observability Ecosystem — Vision Guide

Date: March 26, 2026

Status: Vision Complete / Research Harvested / Ready for Execution

Depends On: GSD-OS Desktop Vision, GSD Chipset Architecture, Amiga New Workbench Vision

Context Windows: 18–24 across 6–8 sessions

Activation Profile: Fleet (20 roles)

1.1 Vision

The Amiga 500 shipped with a custom chipset that embarrassed workstations costing ten times its price. It did not win by having more transistors. It won because Agnus, Denise, and Paula each owned their slice of the machine and communicated through DMA channels that kept the CPU free to think. The result was a computer that could play four-channel audio, animate 32-color sprites, and receive input simultaneously at 7 MHz — a feat that would have required three dedicated processors in any conventional architecture of the era. This is the Amiga Principle made silicon: architectural leverage over brute force.

The GSD-OS desktop is an application of that same principle at the software layer. The CRT shader engine is the Denise chip — it owns the display and renders pixel-perfect phosphor bloom without asking permission. The skill-creator is Agnus — it watches memory usage patterns and pre-stages the next wave before the current one lands. The observability stack is Paula — it listens to everything, reports I/O health, and knows when a channel is saturated before the system does. The mission documented here exists to build the *complete wiring harness* connecting these layers: from the sub-pixel OpenGL fragment shader to the Prometheus scrape interval, from the QPU assembler kernel on a Raspberry Pi 5 to the Grafana dashboard alert that fires when GPU temperature exceeds the ambient thermal envelope of a PNW server room.

The 1001 examples taxonomy is not hyperbole. The NeHe legacy produced 48 canonical OpenGL lessons that taught a generation. LearnOpenGL.com extended that to 60+ modern core-profile chapters. The Khronos Vulkan samples repository contains another 100+ production-quality demonstrations. Shadertoy alone hosts hundreds of thousands of GLSL fragment shaders, of which XScreenSaver 6.14 has begun pulling the open-licensed subset into its 240+-hack ecosystem. The GSD mission is to create a *curated, annotated, cross-referenced* library of 1001 examples organized not as a flat list but as a unit circle of complexity — from the zero-degree “Hello Triangle” through the full rotation of ray marching, compute pipelines, and real-time telemetry rendering.

The Amiga expansion principles thread through the entire architecture as philosophy made hardware. Zorro II's autoconfig protocol — where expansion cards announce their own memory maps at boot — is conceptually identical to GSD skill-creator's autoload system, where skills register themselves into the active context on demand. The trapdoor slot that passed 68000 bus signals through to an accelerator card is the same idea as GSD's DACP three-part bundle: intent + data + executable, a structured handoff where the receiver knows exactly what to do without guessing. Every design choice in this mission should ask: *does this card slot cleanly into the bus, or does it require a firmware patch to work?*

1.2 Problem Statement

1. **The NeHe gap.** NeHe's 48-lesson curriculum was written for OpenGL 1.x immediate mode (1997–2012). LearnOpenGL.com covers modern core-profile but stops at OpenGL 4.3 practices, missing compute shaders, mesh shaders, ray tracing extensions, and Vulkan integration. No single resource unifies NeHe's pedagogical clarity with Vulkan's explicit GPU control model. Developers transitioning from tutorials to production encounter a cliff.

2. **CUDA lock-in on non-NVIDIA silicon.** The Raspberry Pi 5 ships with a VideoCore VII GPU that supports OpenGL ES 3.1 and Vulkan 1.2, yet has no CUDA support and limited OpenCL coverage. Audio DSP workloads that could exploit parallel SIMD lanes on the QPU instead run on a single ARM Cortex-A76 core because no practical guide maps CUDA patterns to equivalent OpenCL/Vulkan compute or ARM NEON intrinsics. The SoC audio integration story is fragmented across five incompatible documentation ecosystems.

3. **Amiga expansion knowledge is dying.** The Zorro bus specification, DMA arbitration protocol, autoconfig ROM mechanism, and chipset coprocessor design patterns that powered the 1985–1994 Amiga expansion ecosystem exist primarily in the Hardware Reference Manual, scattered forum posts, and the memories of engineers who are retiring. The homebrew community building FPGA accelerators (Vampire), new Zorro cards (AmigatronicZ2, Matthias Heinrichs ZorroLAN), and MiSTer FPGA cores is re-deriving this knowledge expensively. A structured reference would halve their iteration cycles.

4. **XScreenSaver as unexploited GSD-OS render canvas.** XScreenSaver 6.14 contains 240+ OpenGL/GLSL screensaver hacks, a Shadertoy-compatible API wrapper, and a virtual root window system that makes the entire desktop a render target. GSD-OS's Tauri shell has no integration with this ecosystem. The CRT shader engine uses a WebGL pipeline that is structurally identical to XScreenSaver's GLSL wrapper but shares no code, documentation, or example library. This is a duplication of effort across two systems that want to do the same thing.

5. **Observability dark spots in GPU workloads.** OpenTelemetry reached GA status for all signals in 2025. Prometheus 3.0 introduced resource attribute promotion for OTel metrics. NVIDIA DCGM Exporter surfaces GPU utilization, temperature, memory, and power as Prometheus metrics. Yet no unified GSD mission documents the wiring from a Raspberry Pi VideoCore temperature sen-

sor through an OTel Collector configuration to a Grafana Alloy pipeline and into a GSD dashboard panel. The hardware exists; the plumbing specification does not.

1.3 Core Concept

The GPU as a specialized DMA coprocessor. Just as Agnus owned chip RAM and the blitter owned the pixel pipeline, the GPU owns the framebuffer and the shader execution units. Every architecture decision in this mission follows from treating the GPU not as "the CPU but parallel" but as a *dedicated co-processor with its own instruction set, memory bus, and DMA engine*. The CPU submits work orders; the GPU executes them asynchronously; the observability stack reads the health telemetry without interrupting either.

1.3.1 Domain Architecture Map

```
GSD-OS Desktop (Tauri Shell) | +-- Blueprint Editor (Canvas) ----------->
WebGL / OpenGL ES 3.1 | | | +-- CRT Shader Engine (Denise) ----> GLSL
Fragment Shaders | +-- Skill-Creator Feed (Agnus) ----> Live Metrics →
Grafana Panel | +-- XScreenSaver Hacks (idle render) ---> GLX Virtual Root
Window | | | +-- Shadertoy-compatible API -------> 240+ GLSL Hacks | +--
Really Slick / OpenGL extras --> GLX Port | +-- Compute Layer | | | +--
Desktop GPU (NVIDIA/AMD) ------> CUDA / Vulkan Compute | +-- Raspberry Pi
5 (VideoCore VII) -> OpenCL (VC4CL) / QPU ASM | +-- Audio SoC DSP pipeline
-------> ARM NEON / OpenCL kernels | +-- Observability Pipeline (Paula) |
+-- NVIDIA DCGM Exporter ----------> Prometheus scrape endpoint +-- OTel
Collector (Alloy) --------> OTLP → Prometheus → Grafana +-- Raspberry Pi
hw metrics -------> node_exporter + custom + - - GSDskill - creatormetrics - - - -- >
customOTelSDKinstrument
```

1.3.2 Module Architecture

```
MODULE 01: OpenGL/Vulkan Foundations (1001 Examples Taxonomy) -> Hello
Triangle → Core Profile → Compute Shaders → Vulkan → SPIR-V
MODULE 02: Compute Without NVIDIA (CUDA → OpenCL/Vulkan/NEON) -> CUDA
concepts → OpenCL equivalents → Vulkan Compute → Pi QPU → Audio DSP
MODULE 03: Amiga Expansion Principles (Zorro Bus + Chipset Architecture)
-> Zorro II/III autoconfig → DMA arbitration → FPGA/Vampire → GSD mapping
MODULE 04: XScreenSaver Engineering (GLX + GLSL + Shadertoy) -> virtual
root window → GLSL portability → 240+ hack catalog → GSD-OS hooks
MODULE 05: GSD-OS Deep Integration -> Tauri shell → Blueprint Editor →
WebGL CRT → skill-creator canvas
MODULE 06: Observability Stack (OTel + Prometheus + Grafana) -> DCGM
exporter → OTel Collector → Alloy → Grafana dashboards → alerts
```

1.4 Research Modules

1.4.1 Module 01 — OpenGL/Vulkan Foundations: 1001 Examples Taxonomy

The canonical learning ladder: NeHe's 48 lessons (immediate mode, historical context) → LearnOpenGL.com's 60+ modern core-profile chapters → open.gl (Alexander Overvoorde) → The Official Vulkan Tutorial (vulkan-tutorial.com) → vkguide.dev (dynamic rendering focus) → Khronos Official Vulkan Samples (100+) → Sascha Willems'

Vulkan samples → Shadertoy GLSL library. This module catalogs all 1001 examples into a unit circle taxonomy: $\theta = 0$ (Hello Triangle) through $\theta = 2\pi$ (deferred rendering with ray tracing extensions). Each example is tagged with: API level (GL 2.1 / GL 4.6 / Vulkan 1.3), technique category, GSD-OS applicability, and Pi-5 compatibility.

1.4.2 Module 02 — Compute Without NVIDIA: CUDA to OpenCL/Vulkan/NEON

CUDA parallel programming patterns translated to their cross-vendor equivalents. The Raspberry Pi 5 VideoCore VII supports Vulkan 1.2 and OpenGL ES 3.1; the older VideoCore IV has VC4CL implementing OpenCL 1.2 embedded profile (approx. 24 GFLOPS). ARM NEON SIMD intrinsics provide 4× speedup on quad-core A76 without any GPU involvement. Audio DSP workloads (FFT, convolution reverb, pitch shifting) map naturally to OpenCL kernels or NEON vectorized C. This module documents the translation table from CUDA kernel patterns to OpenCL C / Vulkan Compute GLSL / ARM NEON intrinsics.

1.4.3 Module 03 — Amiga Expansion Principles: Zorro Bus and Chipset Architecture

The Zorro II bus (Amiga 2000) is a buffered 16-bit extension of the 68000 bus with DMA bus mastering. Zorro III (Amiga 3000/4000) extends to 32-bit with full address decode. The autoconfig mechanism allows cards to self-describe their memory requirements on boot — a 1985 plug-and-play architecture. The three custom chips (Agnus/Denise/Paula) each own dedicated DMA channels, executing concurrently with zero CPU overhead. Modern FPGA accelerators (Apollo/Vampire, MiSTer) re-implement these specs in programmable silicon. This module extracts the design principles that map directly onto GSD's chipset YAML (Agnus=orchestration, Denise=display, Paula=I/O) and DACP structured handoff architecture.

1.4.4 Module 04 — XScreenSaver Engineering: GLX, GLSL, and Shadertoy

XScreenSaver 6.14 (released January 2026) includes 18 new Shadertoy-derived hacks, a Shadertoy-compatible GLSL wrapper ("jwzgles" — OpenGL 1.3 over OpenGL ES, "jwxyz" — X11 over Cocoa), and a virtual root window system where any X11 program can become a screensaver. The hack framework provides windowing, command-line parameter parsing, and preference saving, letting developers focus entirely on the render loop. The Really Slick Screensavers (RSS) GLX port provides another 10 high-quality OpenGL hacks. The GSD-OS CRT shader engine uses a structurally identical GLSL pipeline. This module documents the XScreenSaver hack API, GLSL portability matrix (GL 2.1 / GL 4.6 / ES 3.0 / WebGL 2.0), and integration path into GSD-OS as an idle-state render mode.

1.4.5 Module 05 — GSD-OS Deep Integration

The Tauri-based GSD-OS desktop shell connects the Blueprint Editor (canvas), the skill-creator feed, and the CRT shader engine. This module documents: the WebGL pipeline inside the Tauri WebView, the Bridge between Tauri Rust core and the JS render loop, the event bus that carries skill-creator metrics to the canvas, and the hook points for injecting XScreenSaver-compatible GLSL programs as idle-state wallpapers. The GSD chipset YAML (Agnus/Denise/Paula/68000/Gary) is formalized here as a first-class configuration artifact with version-controlled defaults.

1.4.6 Module 06 — Observability Stack: OpenTelemetry, Prometheus, and Grafana

Full-stack GPU observability from hardware sensor to dashboard alert. NVIDIA DCGM Exporter surfaces GPU metrics (utilization, temperature, power, memory) in Prometheus format. VC4CL and VideoCore VII on Pi expose temperature and utilization via sysfs. The OTel Collector (Grafana Alloy, announced GrafanaCON 2024, Grafana Agent EOL November 2025) pipelines all metrics with OTLP, leveraging Prometheus 3.0's resource attribute promotion (GA, Grafana Cloud default). GSD skill-creator is instrumented with the OTel SDK to emit custom spans. Final output: 5 pre-built Grafana dashboards covering GPU health, Raspberry Pi SoC, audio DSP latency, GSD skill metrics, and the full observability pipeline health.

1.5 Chipset Configuration

```
GSD Chipset YAML — GPU/GSD-OS/Observability Mission chipset: agnus:
Orchestration / DMA / Context model: claude-opus-4-6 role: FLIGHT + INTEG
+ ARCH responsibilities: - Cross-module synthesis and dependency graph
management - Amiga expansion principle extraction and GSD mapping - Wave
gate approval; context budget management allocation: 30
denise:  Display / Output Rendering model: claude-sonnet-4-6 role: EXEC-1
+ EXEC-2 + CRAFT-GL + CRAFT-OBS responsibilities: - OpenGL/Vulkan examples
taxonomy compilation (Module 01) - XScreenSaver hack catalog and GLSL
portability matrix (Module 04) - Observability stack documentation (Module
06) - GSD-OS WebGL pipeline specification (Module 05) allocation: 60
paula:  Audio / I/O / Anticipatory model: claude-haiku-4-5 role: SCOUT +
LOG + BUDGET responsibilities: - Source index and shared schema generation
(Wave 0) - Token budget tracking and session health telemetry - Commit
messages and milestone summaries allocation: 10
```

1.6 Scope Boundaries

1.6.1 In Scope (v1.0)

- OpenGL 2.1 through 4.6 core profile examples (annotated NeHe modernizations + LearnOpenGL curriculum)
- Vulkan 1.2/1.3 introduction through deferred rendering pipeline
- GLSL / SPIR-V shading language reference with Shadertoy compatibility notes
- CUDA-to-OpenCL/Vulkan-Compute pattern translation table
- Raspberry Pi VideoCore VII (Pi 5): Vulkan 1.2, OpenGL ES 3.1, VC4CL (Pi 3/4)
- ARM NEON SIMD audio DSP patterns (FFT, convolution, pitch)
- Amiga Zorro II/III bus specification and autoconfig mechanism
- Amiga custom chipset DMA architecture (Agnus/Denise/Paula coprocessor model)
- Modern Amiga FPGA accelerators: Apollo/Vampire, MiSTer, AmigatronicZ2, ZorroLAN-IDE
- XScreenSaver 6.14 hack framework API (GLX virtual root window, GLSL wrapper)
- Shadertoy-compatible module integration for GSD-OS idle render
- Tauri shell → WebGL bridge hook points
- GSD chipset YAML formalization and version-controlled defaults
- NVIDIA DCGM Exporter → Prometheus scrape configuration

- Grafana Alloy OTel Collector pipeline (OTLP + Prometheus resource attribute promotion)
- 5 pre-built Grafana dashboard specifications
- GSD skill-creator OTel SDK instrumentation hooks

1.6.2 Out of Scope

- DirectX 12 / Metal / WebGPU (separate mission)
- NVIDIA ray tracing hardware (RTX) deep dive — referenced, not primary
- AmigaOS 3.x/4.x application development
- Machine learning / AI inference on GPU (separate GSD mission)
- Production Kubernetes deployment of the observability stack
- Windows-specific screensaver (.scr) API

1.7 Success Criteria

1. A 1001-example taxonomy is compiled, tagged, and cross-referenced with API level, technique category, Pi-5 compatibility, and GSD-OS applicability. Every example has a source URL or repository reference.
2. The NeHe gap is bridged: all 48 NeHe lessons have a modern GL 4.x or Vulkan equivalent documented with GLSL/SPIR-V shader source.
3. The CUDA-to-OpenCL translation table covers at minimum: parallel reduction, FFT, matrix multiply, convolution, histogram, prefix scan. Each entry specifies API, kernel signature pattern, and performance characteristics on Pi 5.
4. Raspberry Pi 5 audio DSP pipeline is documented end-to-end: ALSA/JACK input → ARM NEON or OpenCL kernel → Vulkan compute → audio output with measured latency in milliseconds.
5. Amiga Zorro II/III autoconfig protocol is fully documented with a GSD architectural mapping table (Zorro concept → GSD DACP equivalent).
6. Amiga chipset DMA coprocessor model (Agnus/Denise/Paula) is mapped 1:1 onto GSD chipset YAML roles with rationale for each assignment.
7. XScreenSaver hack framework API is documented with a working Hello-Triangle equivalent hack skeleton and GLSL portability matrix across GL 2.1, GL 4.6, ES 3.0, and WebGL 2.0.
8. A curated 50-hack subset of XScreenSaver/Shadertoy is cataloged for GSD-OS idle render, each annotated for aesthetic category and GPU load.
9. GSD-OS Tauri integration points are specified: WebGL context creation, GLSL program injection, skill-creator metric event bus, and idle-state render mode activation.
10. NVIDIA DCGM Exporter + OTel Collector (Grafana Alloy) pipeline is documented with complete YAML configuration from scrape endpoint to Prometheus remote write.
11. Raspberry Pi SoC temperature and VideoCore utilization metrics are integrated into the same OTel pipeline.

12. Five Grafana dashboard specifications are produced: GPU Health, Pi SoC, Audio DSP Latency, GSD Skill Metrics, and Pipeline Health.

1.8 Relationship to Other GSD Documents

Document	Relationship	Dependency
GSD-OS Desktop Vision	Parent vision; defines Tauri shell, CRT shader engine	Upstream
GSD Chipset Architecture Vision	Agnus/Denise/Paula model formalized here	Upstream
GSD Amiga New Workbench Vision	Workbench UI aesthetic depends on XScreenSaver idle renders	Downstream
GSD Staging Layer Vision	DACP bundles carry GLSL programs between agents	Peer
Deep Audio Analyzer Mission	Audio DSP pipeline (Module 02) directly informs DAA signal chain	Peer
Skill Creator Wasteland Integration	OTel instrumentation of skill-creator (Module 06)	Peer
Fox Infrastructure Group (FoxCompute)	Ocean compute nodes will run the Vulkan Compute workloads	Downstream

1.9 The Through-Line

The Amiga shipped with a chassis that looked like a toy and a chipset that looked like a miracle. The miracle was not the hardware. The miracle was the decision to give each coprocessor its own lane of the bus and trust it to drive its own DMA without supervision. The CPU was freed to do what only the CPU could do: think. Everything else ran in the spaces between clock cycles.

GSD-OS is a software architecture built on the same decision. The shader engine owns the display lane. The skill-creator owns the context lane. The observability stack owns the telemetry lane. Each runs its own DMA equivalent — its own asynchronous event loop — without blocking the others. The 1001 examples in this mission are not a curriculum. They are the unit circle of what those lanes can carry: from a single shaded triangle at $\theta = 0$ to a full deferred renderer with live Grafana telemetry at $\theta = 2\pi$. The spaces between each example are where the real learning happens.

STAGE 2 — RESEARCH REFERENCE

2 GPU Graphics, Compute & GSD-OS Observability — Research Reference

2.1 How to Use This Document

This reference is organized by module. Each section contains sourced findings, quantitative data, and direct references to authoritative repositories. Agents executing Wave 1 should treat each module section as their primary source document.

Key source organizations:

- Khronos Group (khronos.org, vulkan.org, opengl.org) — OpenGL and Vulkan specifications
- The Khronos Group Vulkan Samples Repository (github.com/KhronosGroup)
- LearnOpenGL.com (Joey de Vries) — Modern OpenGL core-profile curriculum
- vulkan-tutorial.com — Canonical Vulkan introduction
- vkguide.dev — Dynamic rendering-focused Vulkan guide
- XScreenSaver (jwz.org) — Jamie Zawinski's screensaver ecosystem
- Raspberry Pi Foundation (raspberrypi.com) — VideoCore GPU documentation
- NVIDIA DCGM (docs.nvidia.com/datacenter/dcgm) — GPU metric exporters
- Grafana Labs (grafana.com) — Alloy, Prometheus, Loki, Tempo
- AMD GPUOpen (gpuopen.com) — Vulkan best practices and profiling

2.2 Module 01 Reference — OpenGL/Vulkan Foundations

2.2.1 The NeHe Legacy and the Modern Gap

NeHe Productions (nehe.gamedev.net, 1997–2012) published 48 lessons covering OpenGL 1.x immediate mode. The site remains archived and widely referenced as pedagogical history. The core gap: immediate-mode functions (glBegin, glEnd, glLight) are deprecated in OpenGL 3.2 core profile and unavailable in OpenGL ES 3.x or WebGL.

LearnOpenGL.com (learnopengl.com) fills this gap with 60+ modern core-profile chapters, available as both a free website and a commercial print book. The curriculum covers: Getting Started → Lighting → Model Loading → Advanced OpenGL → Advanced Lighting → PBR → In Practice → Guest Articles. The site has been continuously revised over 7 years.

The Vulkan Tutorial (vulkan-tutorial.com) by Alexander Overvoorde is the canonical Vulkan introduction, requiring C++ with RAII familiarity and C++17 compiler support (GCC 7+, Clang 5+, MSVC 2017+). It explicitly does not assume OpenGL knowledge but does require 3D graphics fundamentals.

1001 Examples Source Breakdown:

Source	Count	API Level	Pi-5 Compat
NeHe Productions (modernized)	48	GL 2.1–4.x	Partial
LearnOpenGL.com chapters	65	GL 3.3–4.6	ES 3.1 subset
open.gl (Overvoorde)	30	GL 3.x	Yes (ES 3.1)
vulkan-tutorial.com	25	Vulkan 1.2+	Yes (VK 1.2)
vkguide.dev	30	Vulkan 1.3	Yes (VK 1.2)
Khronos Official Vulkan Samples	120	Vulkan 1.x	Partial
Sascha Willems Vulkan Samples	140	Vulkan 1.x	Partial
XScreenSaver GL Hacks	80	GL 1.3–4.x	GL ES 3.0
Shadertoy (open-licensed)	200	GLSL / WebGL	WebGL 2.0
Really Slick Screensavers (GLX)	10	GL 1.x–2.x	Partial
Custom GSD-OS examples	253	Mixed	Yes
Total	**1001**	—	—

2.2.2 GLSL Version Portability Matrix

XScreenSaver 6.14 author Jamie Zawinski documented the portability challenge directly: targeting four incompatible GLSL versions — Linux GL 3.1–4.6 compatibility profile, macOS GL 4.1 core profile, iOS OpenGL ES 3.0, and Android OpenGL ES 3.0–3.2. He implemented compatibility shims to bridge GL ES 3.0 code to GL 2.1 systems.

Feature	GL 2.1	GL 4.6	ES 3.0/WebGL2	Vulkan SPIR-V
Compute Shaders	No	Yes	No	Yes
Geometry Shaders	No	Yes	No	Yes
Tessellation	No	Yes	No	Yes
Integer Types	Limited	Yes	Yes	Yes
Mesh Shaders	No	Ext	No	Ext
Ray Tracing	No	Ext	No	Ext
Bindless Textures	No	Ext	No	Yes

2.3 Module 02 Reference — Compute Without NVIDIA

2.3.1 Raspberry Pi GPU Landscape

The Raspberry Pi 3 Model B+ uses a Broadcom BCM2837B0 with VideoCore IV GPU. VC4CL (VC for Compute Language), implemented by Daniel Steadelmann at Nuremberg Tech, provides OpenCL 1.2 embedded profile support. The VideoCore IV's maximum compute performance is approximately 24 GFLOPS — insufficient for neural network training but adequate for real-time DSP workloads.

The Raspberry Pi 5 ships with Broadcom BCM2712 and VideoCore VII, supporting Vulkan 1.2 natively. Neither VideoCore generation supports CUDA (NVIDIA proprietary) or the full OpenCL specification.

The critical alternative path: ARM NEON SIMD intrinsics via `#pragma omp parallel for` provide 4× speedup on quad-core A76 processors without GPU involvement. For audio DSP workloads, NEON vectorized FFT (e.g., KissFFT, FFTW) often outperforms VideoCore IV OpenCL due to DMA transfer overhead.

2.3.2 CUDA to Cross-Vendor Translation

CUDA Concept	OpenCL Equiv.	Vulkan Compute	Pi-5 / NEON
cudaMalloc	clCreateBuffer	VkBuffer + VkDeviceMemory	malloc + posix_memalign
cudaMemcpy	clEnqueueCopy	vkCmdCopyBuff	memcpy / DMA
__global__ kernel	__kernel function	layout(local_size comp shader	NEON intrinsic loop
blockDim / gridDim	get_global_id()	gl_GlobalInvocat	omp parallel for
cudaDeviceSync	clFinish	vkQueueWaitIdle	pthread_join
shared memory	__local memory	shared (workgroup storage)	stack-allocated array
warp (32 threads)	wavefront (64/32)	subgroup	NEON vector lane

SYCL (C++ abstraction over OpenCL) and Intel oneAPI/DPC++ represent the current state-of-the-art in CUDA portability. ROCm/HIP translates CUDA code to run on AMD GPUs. For embedded SoC work, Vulkan Compute provides the most consistent path across VideoCore VII, Mali, and Adreno GPUs.

2.3.3 Audio DSP on SoC

Key audio processing workloads and their vectorization strategies on Pi 5:

- **FFT (Fast Fourier Transform):** FFTW with NEON backend; alternatively Vulkan Compute FFT via VkFFT library
- **Convolution Reverb:** Frequency-domain multiplication (FFT × IR); NEON float32x4 multiply-accumulate
- **Pitch Shifting:** Phase vocoder on FFT bins; parallelizes cleanly across NEON

lanes
- **Parametric EQ / IIR filters:** Direct Form II transposed; 4-sample NEON pipeline
- **Granular Synthesis:** Embarrassingly parallel grain rendering; ideal Vulkan Compute candidate

Measured latency target for real-time audio on Pi 5: $<$ 10ms round-trip with JACK audio server, 128-frame buffer at 44100 Hz.

2.4 Module 03 Reference — Amiga Expansion Principles

2.4.1 Zorro Bus Architecture

Zorro II (Amiga 2000) is a buffered 16-bit extension of the Motorola 68000 bus with DMA bus mastering support. Zorro III (Amiga 3000/4000) extends to 32-bit addressing and full address decode. The Amiga 4000 provides four Zorro III slots. Fast RAM can be extended via Zorro III cards to 64, 128, or 256 MiB.

The autoconfig mechanism is the architectural gem: each expansion card carries a ROM that announces its memory requirements, interrupt vectors, and device ID to the system on first boot. The OS allocates address space dynamically. No manual jumper configuration required. This is functionally identical to modern PCI-E enumeration — implemented in 1985.

2.4.2 Chipset Coprocessor Architecture

The three Amiga custom chips represent the canonical specialized-coprocessor model:

Chip	Amiga Role	Capabilities	GSD YAML Role
Agnus	DMA controller / Blitter	Owns chip RAM, runs blitter/copper independently	FLIGHT / Orchestration
Denise	Display / Color	Sprites, playfields, color register, genlock	Display / CRT shader
Paula	Audio / I/O	4-channel PCM, disk DMA, serial/parallel I/O	Audio / I/O / Scout
Gary	Glue logic	Address decode, bus arbitration, peripheral enables	68000 / Glue

2.4.3 Modern FPGA Accelerators

The Apollo/Vampire project provides FPGA-based Amiga accelerators with CPU performance beyond the 68060, SAGA graphics at Full-HD resolution, HDMI output, 128 MB RAM, and MicroSD storage. Individual Computers (iComp) produces the iSprint accelerator (MC68EC000 at 14–42 MHz, 8 MB RAM, 8 MB FlashROM). AmigatronicZ2 (Italy) manufactures a Zorro II side-expansion card enabling Amiga 500 owners to use full Zorro II cards. A custom Zorro card by Matthias Heinrichs combining IDE, clock port, and Ethernet has been successfully assembled and benchmarked at 2.8–3 MB/s IDE transfer rates.

2.5 Module 04 Reference — XScreenSaver Engineering

2.5.1 Framework Architecture

XScreenSaver (Jamie Zawinski, 1992–present) is a free and open-source collection of 240+ screensavers for Unix, macOS, iOS, and Android. Version 6.14 (January 2026) added 18 Shadertoy-derived hacks via a new Shadertoy-compatible API wrapper. The framework includes:

- **jwxyz:** Complete X11 API implementation over Cocoa (macOS/iOS)
- **jwzgles:** OpenGL 1.3 API over OpenGL ES 1.0 (iOS/Android)
- **Virtual root window:** Hacks draw to the virtual root window rather than a normal window, enabling them to cover the entire desktop
- **GLX integration:** OpenGL hacks use GLX (not GLFW) to hook into the virtual root window; GLFW is incompatible with the screensaver framework

Writing a new hack requires: including `vroot.h`, using GLX for context creation, implementing the screensaver protocol (handle expose events, periodic redraws, respond to SIGTERM). The existing hack source code in the XScreenSaver distribution provides complete examples.

2.5.2 Shadertoy Licensing Note

Shadertoy's default license is CC-BY-NC-SA (non-commercial, share-alike). This is incompatible with XScreenSaver's open-source license. Only shaders licensed MIT, BSD, CC-BY, CC-BY-SA, or CC0 can be redistributed with XScreenSaver. This licensing constraint shapes the curated GSD-OS 50-hack subset.

2.5.3 50-Hack Curated Subset for GSD-OS

Categories for the GSD-OS idle render catalog:

Category	Count	Representative Hacks	GPU Load
Fractal / Mathematical	8	glmatrix, hypercube, polytopes, glknots	Low–Med
Particle Systems	7	xrayswarm, nerverot, fiberlamp	Low–Med
3D Geometry	9	flipflop, blocktube, topblock, cubestorm	Medium
Amiga-Nostalgic	5	boing, phosphor, starwars, atunnel	Low
Shadertoy GLSL	12	alien beacon, battered planet (new in 6.14)	Med–High
Really Slick Screensavers	5	Skyrocket, Helios, Euphoria, Flux	Medium
Generative Art	4	glslideshow, cloudlife, gflux, xflame	Low

2.6 Module 05 Reference — GSD-OS Deep Integration

Key Tauri integration points for the WebGL/GLSL pipeline:

- **WebView GLSL context:** Tauri's WebView uses the system WebGL2 implementation. On Linux: Mesa with GL 4.x compatibility; on Pi 5: WebGL 2.0 over VideoCore VII
- **GLSL injection path:** Blueprint Editor canvas exposes a `registerShaderProgram(id,`

vertSrc, fragSrc) API callable from Rust core via Tauri commands

- **Skill-creator metric bus:** Tauri event system (emit/listen) carries skill-creator YAML diffs to the canvas as structured JSON; the canvas renders them as animated overlay panels
- **Idle-state render:** When GSD-OS detects > 5 minutes of inactivity, it fires a screensaver:activate event; the canvas switches to the selected GLSL hack at full-screen WebGL
- **CRT shader engine:** The existing phosphor/scanline/bloom fragment shader is the Denise chip of GSD-OS; it runs as a post-processing pass over all canvas output

2.7 Module 06 Reference — Observability Stack

2.7.1 NVIDIA DCGM Exporter Pipeline

NVIDIA's Data Center GPU Manager (DCGM) suite exposes GPU metrics in Prometheus format via dcgm-exporter. Key metrics: DCGM_FI_DEV_GPU_UTIL (utilization %), DCGM_FI_DEV_GPU_ (temperature °C), DCGM_FI_DEV_POWER_USAGE (watts), DCGM_FI_DEV_MEM_COPY_UTIL (memory bandwidth %).

Grafana Cloud's AI Observability integration (GA, February 2025) instruments over 50 gen AI tools via OpenLIT (OpenTelemetry-native SDK) and provides pre-built GPU dashboards tracking utilization, temperature, and power.

2.7.2 Grafana Alloy and Prometheus 3.0

Grafana Alloy (announced GrafanaCON 2024) is the successor to Grafana Agent (EOL November 1, 2025). It is 100% OTLP-compatible and provides native pipelines for both OpenTelemetry and Prometheus formats. Prometheus 3.0 introduced resource attribute promotion: OTel resource attributes (service.name, service.instance.id, k8s.pod.name) are promoted as Prometheus metric labels, eliminating the need for complex target_info joins in PromQL queries.

According to Grafana Labs' 2024 Observability Survey, 89% of respondents invest in Prometheus and 85% in OpenTelemetry, with nearly 40% using both in production.

2.7.3 OTel Collector Configuration Skeleton

```
Grafana Alloy / OTel Collector — GSD-OS GPU Pipeline receivers:
prometheus: config: scrape_configs : -job_name :' dcgm - gpu - metrics'scrape_interval :
15sstatic_configs : -targets : ['localhost : 9400']metric_relabel_configs : -source_labels :
[_name]_regex:'DCGM.*'action:keep-job_name:'pi5-soc-metrics'scrape_interval:30sstatic_configs:-targets:['pi5.local:9100']node_exporter-job_name:'gsd-skill-cre
processors: batch: timeout: 10s send_batch_size : 1024resource : attributes : -action :
insertkey : gsd.environmentvalue : gsd - os - desktop
exporters: prometheusremotewrite: endpoint: "http://prometheus:9090/api/v1/write"
otlphttp/grafana-cloud: endpoint: "https://otlp-gateway.grafana.net/otlp"
headers: Authorization: "Basic GRAFANA_CLOUD_TOKEN"
service: pipelines: metrics: receivers: [prometheus, otlp]
processors: [batch, resource] exporters: [prometheusremotewrite,
otlphttp/grafana-cloud]
```

2.8 Safety and Sensitivity Considerations

Area	Type	Handling
Shader code examples	Technical accuracy	All GLSL/SPIR-V examples must compile without errors on target platform
Open-source licensing	Legal	Shadertoy integration requires CC0/MIT/BSD/CC-BY only; CC-BY-NC-SA explicitly excluded
Security (OTel tokens)	Security	API keys and cloud tokens documented as environment variables, never hardcoded
Raspberry Pi safety	Hardware	QPU assembly notes must include DMA MMU warning (root requirement, memory access risk)
Performance claims	Numerical attribution	All GFLOPS and latency figures attributed to specific benchmarks or datasheets

2.9 Source Bibliography

Khronos Group / Official Specifications

- Vulkan 1.3 Specification — docs.vulkan.org
- OpenGL 4.6 Core Profile Specification — opengl.org
- Khronos Vulkan Samples Repository — github.com/KhronosGroup/Vulkan-Samples
- Khronos Beginners Guide to Vulkan — khronos.org/blog (2017)

Learning Resources (Peer-quality, open curriculum)

- LearnOpenGL.com — Joey de Vries (7-year continuously revised curriculum)
- vulkan-tutorial.com — Alexander Overvoorde (canonical Vulkan introduction)
- vkguide.dev — Victor Blanco (dynamic rendering focus, active 2024)
- open.gl — Alexander Overvoorde (modern OpenGL, interactive examples)

Official Platform Documentation

- Raspberry Pi Documentation — raspberrypi.com/documentation
- Broadcom BCM2712 VideoCore VII Datasheet
- VC4CL: OpenCL for VideoCore IV — github.com/doe300/VC4CL (master thesis, Nuremberg Tech)
- NVIDIA DCGM Documentation — docs.nvidia.com/datacenter/dcgm
- Amiga Hardware Reference Manual — amigadev.elowar.com

Observability Stack

- Grafana Alloy — grafana.com/docs/alloy (GrafanaCON 2024 announcement)
- Prometheus 3.0 Resource Attribute Promotion — grafana.com/blog (2025)
- Grafana AI Observability with GPU Monitoring — grafana.com/whats-new (Feb 2025)
- OpenTelemetry Specification — opentelemetry.io/docs (GA all signals, 2025)

XScreenSaver / Screensaver Engineering

- XScreenSaver 6.14 Release Notes — jwz.org/blog (January 2026)

- XScreenSaver Source Repository — github.com/porridge/xscreensaver
- Writing X11 Screensavers with OpenGL — dbeef.lol (2018, GLX virtual root window)
- Really Slick Screensavers GLX Port — sourceforge.net/projects/rssavers

Amiga Hardware

- Amiga Hardware Database — amiga.resource.cx
- Amiga Zorro II — en.wikipedia.org/wiki/Amiga_Zorro_II
- Building an Amiga Zorro Network Card (GadgetUK, 2025) — theoasisbbs.com
- AMYA500Z2 Zorro II Side Expansion — amigatronic.com / Tindie (2024)
- Apollo/Vampire FPGA Accelerator — apollo-accelerators.com

STAGE 3 — MISSION PACKAGE

3 Milestone Specification

3.1 Mission Objective

Produce a publication-quality reference suite covering OpenGL/Vulkan graphics programming (1001 annotated examples), cross-vendor GPU compute translation, Amiga expansion architecture principles, XScreenSaver/GLSL engineering, GSD-OS deep integration specifications, and a complete OpenTelemetry/Prometheus/Grafana observability stack — all wired together under the Amiga Principle of specialized coprocessors executing in parallel without blocking the CPU.

3.2 Architecture Overview

```
WAVE 0: Foundation (Sequential — Paula/Haiku) Source index, shared
schemas, example taxonomy skeleton, YAML templates ↓ WAVE 1: Parallel
Research (3 Tracks — Denise/Sonnet + Agnus/Opus) Track A: Modules
01+02 (OpenGL/Vulkan + Compute/Pi) [Sonnet] Track B: Modules 03+04
(Amiga + XScreenSaver) [Sonnet + Opus] Track C: Modules 05+06 (GSD-OS
+ Observability) [Sonnet] ── CAPCOM GATE: scope confirm, licensing
review ── ↓ WAVE 2: Synthesis (Sequential — Agnus/Opus) Cross-module
integration; Amiga→GSD mapping tables Unit circle taxonomy finalization;
GLSL portability matrix ── CAPCOM GATE: technical accuracy review ── ↓
WAVE 3: Publication + Verification (Sequential — Denise/Sonnet) LaTeX
compilation, 5 Grafana dashboard specs, index.html OTel config YAML
validation; bibliography audit
```

3.3 Deliverables

#	Deliverable	Acceptance Criteria	Format
D1	1001 Examples Taxonomy	All 1001 entries tagged: API, technique, Pi-5 compat, GSD applicability	Markdown table + JSON
D2	CUDA→OpenCL Translation Table	7+ CUDA patterns with OpenCL/Vulkan/NEON equivalents	Markdown table
D3	Amiga→GSD Mapping Document	Zorro autoconfig → DACP; chipset → chipset YAML	Markdown
D4	XScreenSaver Hack Catalog	50-hack curated subset with GLSL portability notes	Markdown table
D5	GSD-OS Integration Spec	Tauri hook points, GLSL injection API, metric event bus	Markdown
D6	OTel Collector YAML Config	Complete Alloy config, validated syntax	YAML
D7	5 Grafana Dashboard Specs	JSON import files or panel specifications	JSON
D8	Hello-Triangle XScreenSaver Skeleton	Compiles and runs as an XScreenSaver GLX hack	C source
D9	Research Reference PDF	Full three-stage document	PDF/LaTeX

3.4 Component Breakdown

Component	Dependencies	Model	Est. Tokens
Wave 0: Source Index	None	Haiku	8K
Wave 0: Shared Schemas	None	Haiku	5K
Wave 0: Example Taxonomy Shell	Source Index	Haiku	10K
Module 01: GL/VK Foundations	Taxonomy Shell	Sonnet	40K
Module 02: Compute/Pi	Source Index	Sonnet	35K
Module 03: Amiga Architecture	Source Index	Opus	30K
Module 04: XScreenSaver	Source Index	Sonnet	25K
Module 05: GSD-OS Integration	Modules 01,04	Sonnet	30K
Module 06: Observability Stack	Source Index	Sonnet	35K
Wave 2: Amiga→GSD Synthesis	Modules 01–06	Opus	25K
Wave 2: Unit Circle Taxonomy	All modules	Opus	20K
Wave 3: OTel YAML + Dashboards	Module 06	Sonnet	20K
Wave 3: Bibliography Audit	All modules	Sonnet	10K
Wave 3: Final Compilation	All	Sonnet	15K
Total (est.)			**308K**

3.5 Model Assignment Rationale

Role	Model	Budget %	Rationale
Opus (Agnus)	claude-opus-4-6	30%	Synthesis, Amiga architecture judgment, cross-module integration, DACP mapping decisions
Sonnet (Denise)	claude-sonnet-4-6	60%	All six module documents, YAML configs, dashboard specs, compilation
Haiku (Paula)	claude-haiku-4-5	10%	Source index, shared schemas, taxonomy skeleton, budget telemetry

4 Wave Execution Plan

4.1 Wave Summary

Wave	Name	Mode	Model	Outputs
0	Foundation	Sequential	Haiku	Source index, schemas, taxonomy shell
1A	GL/VK + Compute	Parallel	Sonnet	Modules 01, 02
1B	Amiga + XSS	Parallel	Sonnet+Opus	Modules 03, 04
1C	GSD-OS + OTel	Parallel	Sonnet	Modules 05, 06
2	Synthesis	Sequential	Opus	Cross-module maps, unit circle finalization
3	Publication	Sequential	Sonnet	All deliverables, compiled PDF

4.2 Wave 0 — Foundation

Task	Description	Agent	Tokens
Source Index	Compile all authoritative URLs, repos, and documentation references	SCOUT/Hai	3K
Shared Schemas	Define JSON schemas for example entries, translation table rows, hack catalog entries	BUDGET/H	3K
Taxonomy Shell	Create 1001-entry CSV skeleton with column headers; populate first 50 rows	LOG/Haiku	10K

4.3 Wave 1 — Parallel Tracks

Track A — Modules 01 & 02 (CRAFT-GL + CRAFT-COMPUTE / Sonnet):

Task	Description	Tokens
Module 01 draft	Complete NeHe gap analysis; annotate 200+ examples in taxonomy	40K
Module 02 draft	CUDA translation table; Pi 5 audio DSP pipeline docs	35K

Track B — Modules 03 & 04 (CRAFT-AMIGA / Opus + CRAFT-XSS / Sonnet):

Task	Description	Tokens
Module 03 draft	Full Zorro II/III spec, autoconfig, chipset DMA model, FPGA survey	30K
Module 04 draft	XScreenSaver hack API, GLSL portability matrix, 50-hack catalog, Hello-Triangle skeleton	25K

Track C — Modules 05 & 06 (CRAFT-GSD + CRAFT-OBS / Sonnet):

Task	Description	Tokens
Module 05 draft	Tauri hook points, WebGL bridge, GLSL injection API, metric event bus, chipset YAML v2	30K
Module 06 draft	OTel Collector YAML, DCGM exporter config, 5 Grafana dashboard specs	35K

CAPCOM Gate 1: Tibsfox reviews scope and licensing before Wave 2. Key checks: Shadertoy license compliance confirmed; all YAML configs reviewed for security; Amiga architecture claims verified against Hardware Reference Manual.

4.4 Wave 2 — Synthesis

Task	Description	Tokens
Amiga→GSD Mapping	Full mapping table: Zorro → DACP, autoconfig → skill autoload, chipset → YAML roles	20K
Unit Circle Taxonomy	Assign θ values to all 1001 examples; produce visualization spec	20K
GLSL Portability Matrix	Complete cross-version compatibility table with compat shims	15K
Cross-Module Integration	Verify all six modules reference each other correctly; no orphan sections	10K

CAPCOM Gate 2: Technical accuracy review. Tibsfox confirms: unit circle taxonomy assignments make sense; Amiga mapping is architecturally defensible; GSD-OS integration spec is implementable with Tauri 2.x.

4.5 Wave 3 — Publication and Verification

Task	Description	Tokens
OTel YAML Validation	Lint and validate Alloy config; test against OTel Collector contrib schema	8K
Hello-Triangle Skeleton	C source for a compiling XScreenSaver GLX hack with GLSL	10K
Grafana Dashboard JSONs	Five complete dashboard import files	12K
Bibliography Audit	Verify all URLs resolve; confirm license information; flag any 404s	8K
Final LaTeX Compilation	Three-pass xelatex; PDF + .tex source	5K
Index Page	index.html with download cards	5K

4.6 Cache Optimization Strategy

- Wave 0 shared schemas are computed once and injected into all six Wave 1 agent contexts as a shared prefix — exploiting the 5-minute cache TTL
- The source index (URLs, repos, documentation references) is injected into all agents, eliminating redundant web research
- The 1001-example taxonomy shell CSV is computed in Wave 0 and appended to by each Wave 1 track, avoiding the $O(n^2)$ re-read problem
- Modules 03 and 04 share the Amiga Hardware Reference Manual excerpt as a common context block

4.7 Token Budget

Wave	Opus Tokens	Sonnet Tokens	Haiku Tokens
Wave 0	0	0	23K
Wave 1A	0	75K	0
Wave 1B	30K	25K	0
Wave 1C	0	65K	0
Wave 2	65K	0	0
Wave 3	0	48K	0
Total	**95K**	**213K**	**23K**
Split	**29%**	**64%**	**7%**

4.8 Dependency Graph

```
Source Index ──────────────────────┐ Shared Schemas ────────────────┐ │ Taxonomy
Shell ─────────────┐ │ │ │ │ ┌──────────────────┘ │ │ ▼ │ │ Module 01 (GL/VK)
────────┤ │ Module 02 (Compute) ────┤ │ Module 03 (Amiga) ────────┤ │ ← Opus
priority Module 04 (XSS) ──────────┤ │ Module 05 (GSD-OS) ────────┤ │ Module
06 (OTel) ────────┘ │ │ │ ▼ │ Wave 2 Synthesis ──────────────┘ │ ▼ Wave 3
Publication │ ▼ All Deliverables (D1–D9)
```

5 Test Plan

5.1 Test Categories

Category	Purpose	Count	Failure Action	Priority
Safety-Critical	Non-negotiable boundaries	6	BLOCK	Mandatory
Core Functionality	Deliverable acceptance	22	BLOCK	Required
Integration	Cross-module validation	10	BLOCK	Required
Edge Cases	Robustness	7	LOG	Best-effort
Total		**45**		

5.2 Safety-Critical Tests

ID	Verifies	Expected
SC-SRC	Source quality	All citations traceable to Khronos, Raspberry Pi Foundation, Grafana Labs, or equivalent professional sources. Zero forum posts as primary sources.
SC-NUM	Numerical attribution	Every GFLOPS figure, latency measurement, and percentage attributed to a specific benchmark, datasheet, or publication.
SC-LIC	Open-source licensing	All Shadertoy-derived examples verified CC0/MIT/BSD/CC-BY only. No CC-BY-NC-SA content in GSD-OS distribution.
SC-SEC	Security: no hardcoded credentials	OTel config YAML uses environment variable references for all API tokens. Zero literal secrets in any deliverable.
SC-HW	Hardware safety: QPU DMA warning	VC4CL section explicitly documents root privilege requirement and memory management unit absence on VideoCore IV.
SC-ADV	No policy advocacy	Document presents GPU architecture comparisons without vendor advocacy. CUDA/OpenCL comparison presented as technical trade-off analysis.

5.3 Core Functionality Tests (Selected)

ID	Verifies	Expected
CF-01	Example taxonomy completeness	All 1001 entries have: source URL, API level, technique tag, Pi-5 compat, GSD applicability
CF-02	NeHe gap bridged	All 48 NeHe lessons have a modern equivalent with GLSL/SPIR-V source
CF-03	CUDA translation table coverage	$\geq$ 7 CUDA patterns with OpenCL/Vulkan/NEON equivalents
CF-04	Pi 5 audio pipeline documented	End-to-end path from ALSA/JACK to NEON/OpenCL kernel to output with latency spec
CF-05	Amiga autoconfig documented	Zorro II/III autoconfig mechanism fully specified with GSD DACP mapping
CF-06	Chipset mapping complete	Agnus/Denise/Paula each mapped to GSD chipset YAML role with rationale
CF-07	Hello-Triangle hack compiles	XScreenSaver skeleton builds with `gcc` and runs as GLX screensaver
CF-08	50-hack catalog complete	All 50 entries have: name, source, GLSL version, GPU load estimate, license
CF-09	OTel YAML validates	Alloy config lints without errors against OTel Collector contrib schema
CF-10	All 5 dashboards specified	GPU Health, Pi SoC, Audio DSP Latency, GSD Skill Metrics, Pipeline Health

5.4 Verification Matrix

Success Criterion	Test ID(s)	Status
1001 examples tagged and cross-referenced	CF-01, CF-02	Pending
NeHe gap bridged with modern equivalents	CF-02	Pending
CUDA→OpenCL translation table	CF-03	Pending
Pi 5 audio DSP pipeline end-to-end	CF-04	Pending
Amiga autoconfig → DACP mapping	CF-05	Pending
Chipset coprocessor model → GSD YAML	CF-06	Pending
XScreenSaver Hello-Triangle skeleton	CF-07	Pending
50-hack curated GSD-OS catalog	CF-08	Pending
GSD-OS Tauri integration specified	IN-01, IN-02	Pending
OTel pipeline configured end-to-end	CF-09, SC-SEC	Pending
5 Grafana dashboards specified	CF-10	Pending
All cross-module references consistent	IN-03, IN-04	Pending

6 Mission Crew Manifest

Role	Agent	Responsibility	Model
FLIGHT	Orchestrator	Mission direction; wave go/no-go gates; budget approval	Opus
PLAN	Planner	Wave decomposition; dependency graph; parallel track management	Opus
TOPO	Topology Manager	Mid-mission agent restructuring; team-of-teams coordination	Opus
ARCH	Architecture	Cross-cutting decisions: taxonomy schema, YAML config structure	Opus
ANALYST	Pattern Analyst	Unit circle taxonomy θ assignments; cross-module pattern detection	Opus
INTEG	Integration	Cross-module reference validation; Amiga→GSD mapping synthesis	Opus
RETRO	Retrospective	Post-wave analysis; lessons learned for skill-creator	Opus
CRAFT-GL	OpenGL/Vulkan Specialist	Modules 01 & 02 primary author	Sonnet
CRAFT-AMIGA	Amiga Architecture Specialist	Module 03 primary author; Zorro/DMA domain authority	Sonnet
CRAFT-XSS	XScreenSaver/GLS Specialist	Module 04; Hello-Triangle skeleton; GLSL portability	Sonnet
CRAFT-GSD	GSD-OS Integration Specialist	Module 05; Tauri hook points; WebGL bridge	Sonnet
CRAFT-OBS	Observability Specialist	Module 06; OTel YAML; Grafana dashboards	Sonnet
EXEC-1	Executor Track A	Document production: Modules 01, 02	Sonnet
EXEC-2	Executor Track B	Document production: Modules 03, 04	Sonnet
EXEC-3	Executor Track C	Document production: Modules 05, 06	Sonnet
SECURE	Security	License compliance; credential scanning; hardware safety notes	Sonnet
VERIFY	Verification	Source validation; numerical attribution; URL audit	Sonnet
CAPCOM	HITL Interface	Only channel to Tibsfox; gate approvals; scope change requests	Sonnet
SCOUT	Research Agent	Source gathering; web research; filling identified gaps	Haiku
BUDGET	Resource Manager	Token budget tracking; session health telemetry	Haiku
LOG	Chronicle	Commit messages; wave summaries; audit trail	Haiku

7 Execution Summary

Metric	Value
Activation Profile	Fleet (21 roles)
Research Modules	6
Wave Count	4 (Wave 0 + Wave 1 parallel ×3 + Wave 2 + Wave 3)
Parallel Tracks	3 simultaneous in Wave 1
Estimated Context Windows	18–24 across 6–8 sessions
Total Token Budget (est.)	331K (Opus: 95K / Sonnet: 213K / Haiku: 23K)
Model Split	29% Opus / 64% Sonnet / 7% Haiku
CAPCOM Gates	2 (post Wave 1, post Wave 2)
Deliverables	9 (D1–D9)
Total Tests	45 (6 safety-critical / 22 core / 10 integration / 7 edge)
Success Criteria	12
Primary Authorities	Khronos Group, Raspberry Pi Foundation, Grafana Labs, jwz.org

"The Amiga did not win by being the most powerful machine of 1985. It won by being the most ***architecturally honest*** *machine of 1985 — each coprocessor doing exactly the one thing it was built to do, exactly when it was supposed to do it, without interrupting the others. The spaces between their DMA cycles were where the music played."*

This mission follows that architecture from the fragment shader to the Grafana dashboard. Every example in the 1001-example taxonomy is a clock cycle. The understanding lives in the spaces between them. Hand this document to GSD skill-creator and let the coprocessors run.